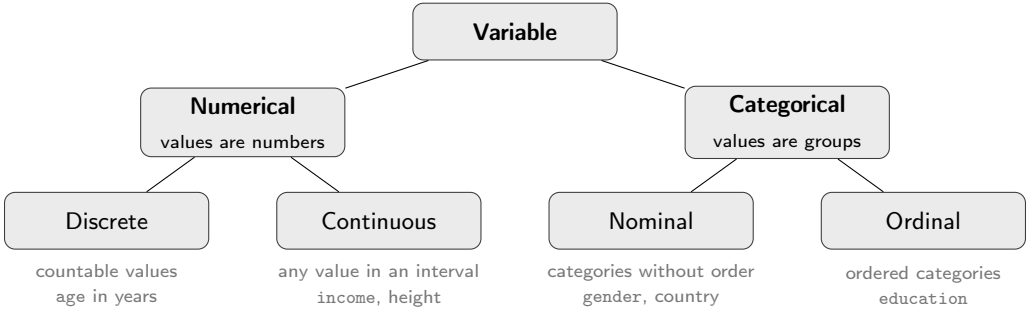
Applied Economics

Data and Plotting

Pello Aspuru

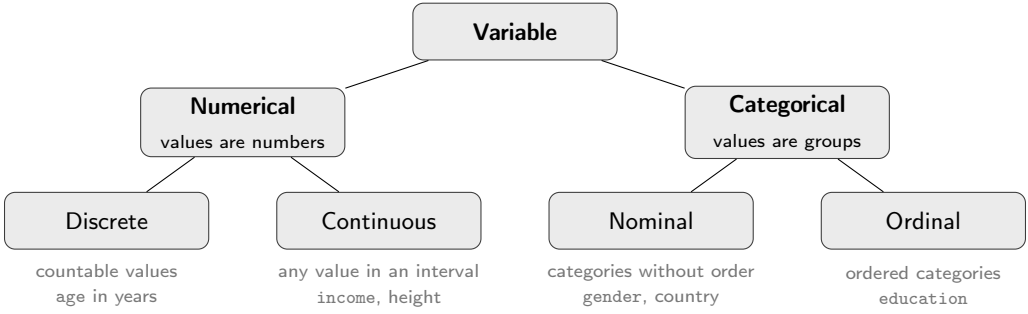
Winter 2026/2027

Types of Variables in a Dataset



- A **binary** (dummy) variable is a categorical variable with two categories, coded as 0 and 1: `employed`.
- In R, `class(df$income)` returns the type of a variable: `numeric` or `integer` for numerical variables, `character` or `factor` for categorical ones.

Types of Variables in a Dataset



- A **binary** (dummy) variable is a categorical variable with two categories, coded as 0 and 1: `employed`.
- In R, `class(df$income)` returns the type of a variable: `numeric` or `integer` for numerical variables, `character` or `factor` for categorical ones.

Importing and Saving Data

- .Rdata

  `load("filename.RData"),`

  `save(object, file = "filename.RData")`

- .csv

  `read.csv("filename.csv"),`

  `write.csv(object, file = "filename.csv")`

- .xls `read_xls("filename.xls")` from package *readxl*

- .dta `read_dta("filename.xls")` from package *haven*

Inspecting Data

- names of variables in your data: `names(...)`
- list the first/last six observations: `head(...)`, `tail(...)`
- summary statistics (mean, max, min and quartiles; variance, st. dev.): `summary(...)`, `var(...)`, `sd(...)`
- frequency table of one variable and cross-table of two variables: `table(...)`, `table(...,...)`
- check for missing data: `is.na(...)`

Accessing data - Selecting Observations and Variables

select a specific variable: `dataframe$var1` or `dataframe[[1]]`

```
df$age # Print the age column of df
df[[2]] # Same as previous line
```

select values in specific rows and columns with indices: `dataframe[..., ...]`

```
# Accessing data through row/column indexes
df[1:3,] # Rows 1 to 3, all columns.
df[,c(1,4)] # All rows, Columns 1 and 4.
df[c(1,25),c(1,3)] # Rows 1 and 25, Columns 1 and 3.
```

`dataframe <- dataframe[...,...]`

→ Danger! drops all observations and variables that are not selected from the data.

`dataframe_new <- dataframe[...,...]`
→ stores the desired subset in a new dataframe, while keeping the original data.

```
df_first_fifty_rows <- df[1:50,]
```

Accessing data - Selecting Observations and Variables

select a specific variable: `dataframe$var1` or `dataframe[[1]]`

```
df$age # Print the age column of df
df[[2]] # Same as previous line
```

select values in specific rows and columns with indices: `dataframe[..., ...]`

```
# Accessing data through row/column indexes
df[1:3,] # Rows 1 to 3, all columns.
df[,c(1,4)] # All rows, Columns 1 and 4.
df[c(1,25),c(1,3)] # Rows 1 and 25, Columns 1 and 3.
```

`dataframe <- dataframe[...,...]`

→ Danger! drops all observations and variables that are not selected from the data.

`dataframe_new <- dataframe[...,...]`
→ stores the desired subset in a new dataframe, while keeping the original data.

```
df_first_fifty_rows <- df[1:50,]
```

Accessing data - Selecting Observations and Variables

select a specific variable: `dataframe$var1` or `dataframe[[1]]`

```
df$age # Print the age column of df
df[[2]] # Same as previous line
```

select values in specific rows and columns with indices: `dataframe[..., ...]`

```
# Accessing data through row/column indexes
df[1:3,] # Rows 1 to 3, all columns.
df[,c(1,4)] # All rows, Columns 1 and 4.
df[c(1,25),c(1,3)] # Rows 1 and 25, Columns 1 and 3.
```

`dataframe <- dataframe[...,...]`

→ Danger! drops all observations and variables that are not selected from the data.

`dataframe_new <- dataframe[...,...]`
→ stores the desired subset in a new dataframe, while keeping the original data.

```
df_first_fifty_rows <- df[1:50,]
```

Accessing data - Selecting Observations and Variables

select a specific variable: `dataframe$var1` or `dataframe[[1]]`

```
df$age # Print the age column of df
df[[2]] # Same as previous line
```

select values in specific rows and columns with indices: `dataframe[..., ...]`

```
# Accessing data through row/column indexes
df[1:3,] # Rows 1 to 3, all columns.
df[,c(1,4)] # All rows, Columns 1 and 4.
df[c(1,25),c(1,3)] # Rows 1 and 25, Columns 1 and 3.
```

`dataframe <- dataframe[...,...]`

→ Danger! drops all observations and variables that are not selected from the data.

`dataframe_new <- dataframe[...,...]`
→ stores the desired subset in a new dataframe, while keeping the original data.

```
df_first_fifty_rows <- df[1:50,]
```

Accessing data - Logical Operators

R can use **logical conditions** to generate logical vectors:

```
# Logical condition vector
index_old_males <- (df$gender == "Male") & (df$age > 50)
table(index_old_males)
```

We can use the logical vector to select the subsample of the dataset:

```
df_old_males <- df[index_old_males, ]
```

Or, directly:

```
df_old_males <- df[(df$gender == "Male") & (df$age > 50), ]
```

Ask ChatGPT: "can you list me the most common logical operations in R"

Accessing data - Logical Operators

R can use **logical conditions** to generate logical vectors:

```
# Logical condition vector
index_old_males <- (df$gender == "Male") & (df$age > 50)
table(index_old_males)
```

We can use the logical vector to select the subsample of the dataset:

```
df_old_males <- df[index_old_males, ]
```

Or, directly:

```
df_old_males <- df[(df$gender == "Male") & (df$age > 50), ]
```

Ask ChatGPT: "can you list me the most common logical operations in R"

Accessing data - Logical Operators

R can use **logical conditions** to generate logical vectors:

```
# Logical condition vector
index_old_males <- (df$gender == "Male") & (df$age > 50)
table(index_old_males)
```

We can use the logical vector to select the subsample of the dataset:

```
df_old_males <- df[index_old_males, ]
```

Or, directly:

```
df_old_males <- df[(df$gender == "Male") & (df$age > 50), ]
```

Ask ChatGPT: "can you list me the most common logical operations in R"

Transforming variables in a Data Frame

To drop *var*1 use:

```
dataframe$var1 <- NULL
```

Generating new variables as transformation of existing variables, e.g.:

```
# New variables generation
df$log_age <- log(df$age)
df$is_higly_educated <- df$education %in% c("Master", "PhD")
```

The variable type of `is_higly_educated` is...

Transforming variables in a Data Frame

To drop *var*1 use:

```
dataframe$var1 <- NULL
```

Generating new variables as transformation of existing variables, e.g.:

```
# New variables generation
df$log_age <- log(df$age)
df$is_higly_educated <- df$education %in% c("Master", "PhD")
```

The variable type of `is_higly_educated` is...

Plotting Data with ggplot2

```
install.packages("ggplot2")
library(ggplot2)
```

```
# Histogram of income
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 30) +
  labs(x = "Income", y = "Frequency")
```

```
# Scatterplot of income against age
ggplot(df, aes(x = income, y = age)) +
  geom_point(size  = 2) +
  labs(x = "Income", y = "Age")
```

Plotting Data with ggplot2

```
install.packages("ggplot2")
library(ggplot2)
```

```
# Histogram of income
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 30) +
  labs(x = "Income", y = "Frequency")
```

```
# Scatterplot of income against age
ggplot(df, aes(x = income, y = age)) +
  geom_point(size  = 2) +
  labs(x = "Income", y = "Age")
```

Plotting Data with ggplot2

```
install.packages("ggplot2")
library(ggplot2)
```

```
# Histogram of income
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 30) +
  labs(x = "Income", y = "Frequency")
```

```
# Scatterplot of income against age
ggplot(df, aes(x = income, y = age)) +
  geom_point(size  = 2) +
  labs(x = "Income", y = "Age")
```

Main Types of Graphs in Economic Research

Which graph to use depends on **how many** variables you want to show and on their **type**:

- **Continuous** variables take many numerical values (income, age, GDP).
- **Categorical** variables take a few values or labels (gender, education, country).

Graph	Variables	Shows	ggplot2
Histogram	1 continuous	distribution of a variable	`geom_histogram()`
Bar chart	1 categorical	number of observations per category	`geom_bar()`
Box plot	1 continuous + 1 categorical	distribution within each category	`geom_boxplot()`
Scatter plot	2 continuous	relationship between two variables	`geom_point()`
Line graph	1 continuous + time	evolution of a variable over time	`geom_line()`

All of them allow for **different categories**: a categorical variable can be added as a colour, a fill, or a separate panel.

Main Types of Graphs in Economic Research

Which graph to use depends on **how many** variables you want to show and on their **type**:

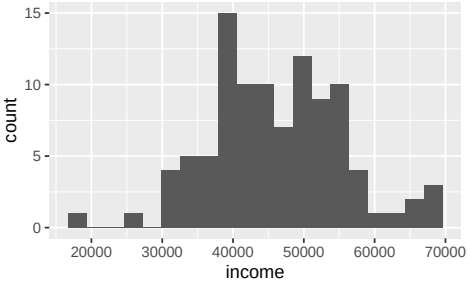
- **Continuous** variables take many numerical values (income, age, GDP).
- **Categorical** variables take a few values or labels (gender, education, country).

Graph	Variables	Shows	ggplot2
Histogram	1 continuous	distribution of a variable	`geom_histogram()`
Bar chart	1 categorical	number of observations per category	`geom_bar()`
Box plot	1 continuous + 1 categorical	distribution within each category	`geom_boxplot()`
Scatter plot	2 continuous	relationship between two variables	`geom_point()`
Line graph	1 continuous + time	evolution of a variable over time	`geom_line()`

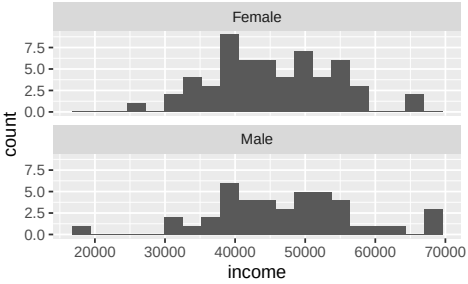
All of them allow for **different categories**: a categorical variable can be added as a colour, a fill, or a separate panel.

Histogram

One continuous variable. Each bar counts the observations in an interval (bin). Categories: one panel per category.



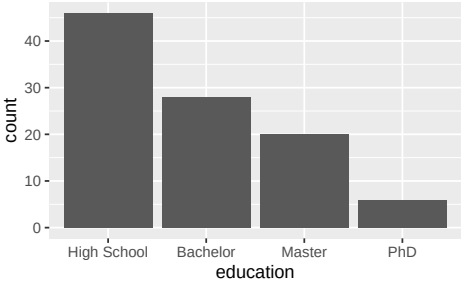

```
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 20)
```



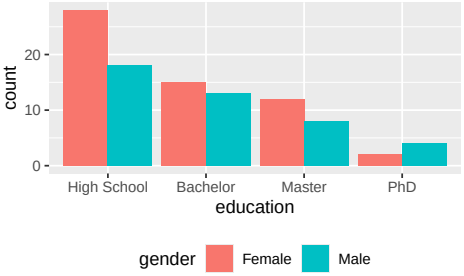

```
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 20) +
  facet_wrap(~ gender, ncol = 1)
```

Bar Chart

One categorical variable. Each bar counts the observations in a category. Categories: one bar per category.



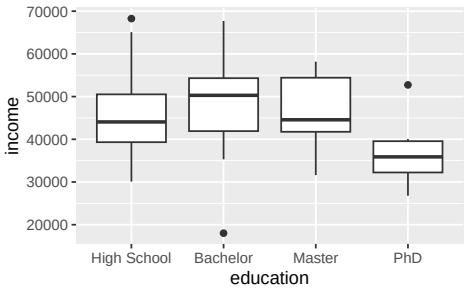

```
ggplot(df, aes(x = education)) +
  geom_bar()
```

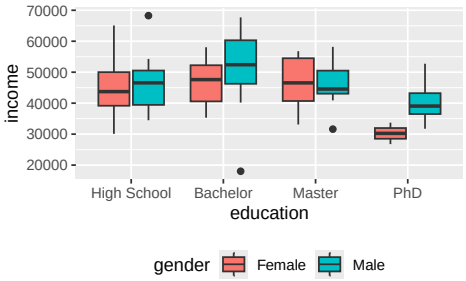



```
ggplot(df, aes(x = education, fill = gender)) +
  geom_bar(position = "dodge") +
  theme(legend.position = "bottom")
```

Box Plot

One continuous and one categorical variable. Each box shows the median and the quartiles of a category. Categories: split each box by a second categorical variable.



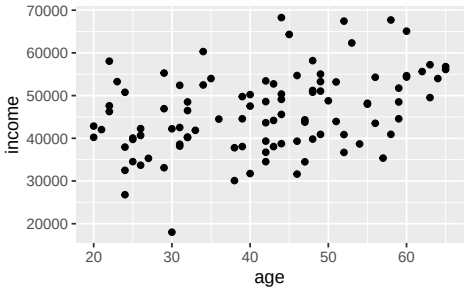


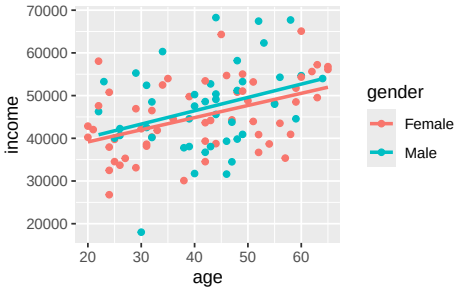

```
ggplot(df, aes(x = education, y = income)) +
  geom_boxplot()
```

```
ggplot(df, aes(x = education, y = income,
               fill = gender)) +
  geom_boxplot() +
  theme(legend.position = "bottom")
```

Scatter Plot

Two continuous variables. Each point is one observation. Categories: one colour (and one fitted line) per category.



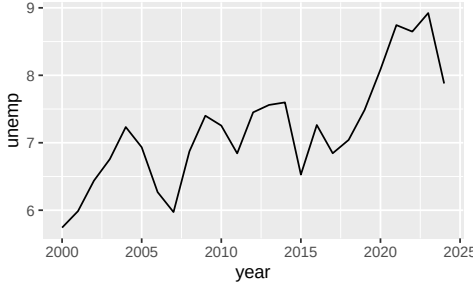



```
ggplot(df, aes(x = age, y = income)) +
  geom_point()
```

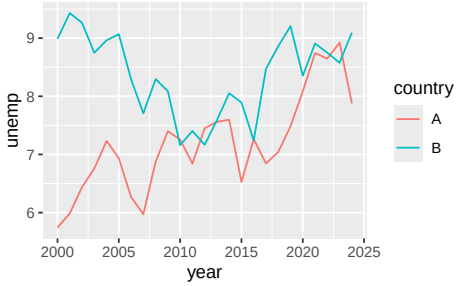
```
ggplot(df, aes(x = age, y = income,
               colour = gender)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```

Line Graph

One continuous variable over time. Points are connected in time order. Categories: one line per category.




```
ggplot(df_ts[df_ts$country == "A", ],
       aes(x = year, y = unemp)) +
  geom_line()
```




```
ggplot(df_ts, aes(x = year, y = unemp,
                  colour = country)) +
  geom_line()
```

Graphs: Summary

Graph	Variables plotted	Adding categories	Variables with categories
Histogram	1	panels (`facet_wrap`) or `fill`	2
Bar chart	1	`fill`	2
Box plot	2	`fill`	3
Scatter plot	2	`colour` or `shape`	3
Line graph	2	`colour` or `linetype`	3

- Panels (`facet_wrap`) work with every graph, and can be combined with colours to add yet another variable.
- Always label the axes and state the units: `labs(x = "...", y = "...")`.
- Later in the course: graphs of **estimates** rather than raw data (fitted regression lines, coefficients with confidence intervals).

Graphs: Summary

Graph	Variables plotted	Adding categories	Variables with categories
Histogram	1	panels (`facet_wrap`) or `fill`	2
Bar chart	1	`fill`	2
Box plot	2	`fill`	3
Scatter plot	2	`colour` or `shape`	3
Line graph	2	`colour` or `linetype`	3

- Panels (`facet_wrap`) work with every graph, and can be combined with colours to add yet another variable.
- Always label the axes and state the units: `labs(x = "...", y = "...")`.
- Later in the course: graphs of **estimates** rather than raw data (fitted regression lines, coefficients with confidence intervals).